

NELSON MANDELA METROPOLITAN UNIVERSITY
SUMMERSTRAND CAMPUS (SOUTH)

SEMESTER TEST QUESTION PAPER

16 MAY 2014

SUBJECT: Advanced Programming 3.1

MODULE CODE: WRAP301

TIME: 3 hours

MARKS: 100

EXAMINERS:

MODERATORS:

1. Dr. D. Vogts

1. Dr. K.A. Naudé

SPECIAL INSTRUCTIONS

1. Answer **ALL** questions.
 2. This question paper consists of 2 sections:
 - Section A: 14 marks **written questions**.
 - Section B: 86 marks Java **practical question**.
 3. Section A must be completed and handed in before Section B may be started.
 4. No permitted literature for Section A.
 5. Permitted materials for Section B:
 - JDK documentation (can be found on C:).
 - Bound lecture notes.
 - Online lecture notes ([//cs2/courses/wrap301/WRAP301 Notes.pdf](http://cs2/courses/wrap301/WRAP301%20Notes.pdf)).
 - Clean Java text book (does not have to be prescribed one, but may not be written in).
 6. Your solutions for the questions in this paper **must** be your own original work.
 7. You must place a copy of your code on the T: drive in the sub-folder allocated to you.
-

Section A: Written Questions (Closed Book)**[14]**

-
- 1) What is the difference between the `paintComponent` and `repaint` methods? (2)
- 2) Consider the top-level and inner class below. (2)
- ```
public class T {
 private int t;

 public void m(int x) {
 int a;

 I i = new I() {
 int a;

 public void f(int e) {
 ...
 }
 };
 }
}
```
- 2.1) Which variable(s) can the `f` method access? (2)
- 2.2) How would the `f` method gain access to the `this` pointer of the outer class? (1)
- 3) What is the difference between a frame and a panel? (2)
- 4) Below is an extract of a program that handles different button click events:
- ```
public class MyApp extends Frame implements ActionListener
{
    Button previouscontact, nextcontact;
    // other controls here

    public void buildUI() {
        add(previouscontact = new Button("Previous Contact"));
        previouscontact.addActionListener(this);

        add(nextcontact = new Button("Next Contact"));
        nextcontact.addActionListener(this);

        // other GUI building code here
    }

    public void actionPerformed(ActionEvent ev)
    {
        Button btnClicked = (Button)ev.getSource();

        if(btnClicked == previouscontact) {
            System.out.println("previous");
        } else if(btnClicked == nextcontact) {
            System.out.println("next");
        }
    }

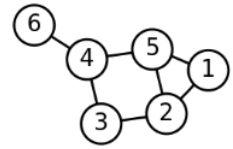
    // rest of class here
}
```
- 4.1) Write the source code to demonstrate how **named inner classes** could be used to handle the previous contact and next contact button clicks individually, by writing the code necessary for the **previous contact button only**, i.e. each button must have its own event handling code and not share the same event handler as above. (3)
- 4.2) Write the source code to demonstrate how **unnamed inner classes** could be used to handle the previous contact and next contact button clicks individually, by writing the code necessary for the **previous contact button only**, i.e. each button must have its own event handling code and not share the same event handler as above. (2)

Hand in Section A to receive Section B

Note: You *may* ask for this section again once you've completed Section B (but may not swap again after that).

Section A: Practical Questions (Open Book)**[86]**

You have encountered the graph data structure in a number of CS modules. It consists of a number of nodes (the circles), which are joined by edges (the lines between the circles). A node may be labelled (the number in the circle). There are other properties of graphs, but these are sufficient for this test.

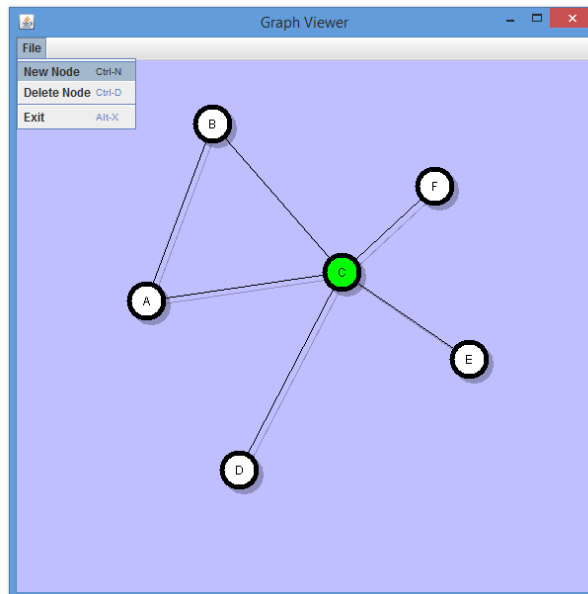


As you have learned, the Model-View-Controller architecture attempts to separate the data structure from the way in which it is represented to the user and the actions that the user may perform to manipulate the data structure by interaction with the representation.

For this question, you will be required to:

- Create a model representing the graph data structure;
- Create a view/controller component that will draw the graph on the screen and allow the user to manipulate the underlying graph by interacting with the component; and finally
- Create an application that uses the model and component you wrote to “put it all together”.

The final application should look something like the screen shot shown below:



You may run the `GraphViewer.jar` application on the test folder to see how the application should function.

- 1) Write a class called *GraphModel* that is a *model* of the graph data structure. For this test, each node will have a label, as well as an (x, y) position describing where it lies on an imaginary two dimensional canvas¹. (24)

It should be possible to:

- add, remove², relabel and move nodes; and
- add and remove edges.

The *GraphModel* should notify interested listeners when something about the graph has changed (so they could, for example, repaint themselves). Therefore:

- provide a *ChangeListener* interface that classes wishing to listen to the model should implement;

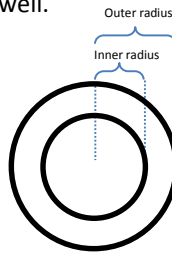
¹ If you were doing this for real, then the model should have absolutely no idea of view-related details, such as position.

² Removing a node removes all edges touching it as well.

- add and remove listeners; and
 - method(s) to *fire* the notifications when something interesting happens to the model.
- 2) Write a *view/controller* class called *GraphView* which is a SWING component that has a model, as described in (1), draws it on the screen and allows the user to interact with it. The component should have the following functionality: (47)
- *left-clicking* on a node will select it;
 - *double left-clicking* on a node will make a text field *appear* that is centred on the node and has the node's label as its text. Typing in a string and then pressing enter will set the node's label as the text field's text and thereafter will make the text field *disappear*;
 - performing a *pop-up trigger* on an empty space (not on a node), will cause a pop-up menu to appear. The menu will have a single menu item "Add Node". Selecting this menu item will cause a new node to be added at the cursor position and have the label "?";
 - performing a *left* mouse-press, drag and release starting on a node will perform a drag-and-drop operation on the node;
 - performing a *right* mouse-press, drag and release starting and ending on different nodes will drag out a visible line between the nodes and will create an edge between the nodes once the mouse button is released.

The component will paint itself in the following manner:

- the background of the component must be the colour (0.75, 0.75, 1.0);
- an edge is drawn as a black line between the two nodes to which it is connected and has a drop-shadow, colour (0, 0, 0, 0.25), offset 5 pixels to the right and 5 pixels below it;
- an edge being dragged out is drawn in red and also has a drop shadow;
- a node is drawn as a circle with outer radius 20 and inner radius 15, with the label centred in the circle. A node has a drop shadow as well.



- an unselected node has a black outer circle and a white inner circle;
- a selected node has a black outer circle and a green inner circle;
- a node from which an edge is being dragged has a red outer circle; and
- a node under the end-point of an edge being dragged out has a red outer circle.

Question 3 on next page...

3) Write an application that:

(15)

- has a size of 600×600 and be placed at (100, 200) on the screen;
- has a menu bar at the top and a *GraphView* filling the JFrame's area;
- has its title set to "Graph Viewer";
- the menu bar must have a menu called "File", which has items:
 - "New Node", accelerator CTRL+N, which adds a new node to the graph at position (0, 0);
 - "Delete Node", accelerator CTRL+D, which deletes the currently selected node;
 - Separator; and
 - "Exit", accelerator ALT+X, which exits the application.

Once completed:

- **Remember to put your name and student number at the top of each .java file.**
- **SAVE your work to the T:**
- **Remember that if you do not save all your work to the T:, there is nothing to mark, i.e. expect 0!**
- **You may not leave the venue until the last 15 minutes of the test.**
- **You *may* ask for Section A again once you have handed in this section, but may *not* ask for this section again after that.**